

The Barnes-Hut Gravity Tree on an FPGA

ANGUS BEANE ^{1,2} AND OTHERS³

¹Center for Cosmology and Particle Physics, Department of Physics, New York University, 726 Broadway, New York, NY 10003, USA

²Simons Society of Fellows, Simons Foundation, New York, NY 10010, USA

³Places

ABSTRACT

Directly solving for the gravitational force at a given position by a collection of discrete particles has an $\mathcal{O}(N^2)$ complexity, making the exact solution intractable for problems which require large numbers of particles. The Barnes-Hut (BH) tree is one of several approximate methods which can achieve $\mathcal{O}(N \log N)$ complexity or better, enabling simulations of galaxies, cosmological boxes, star clusters, and many others where self-gravity is important. However, the algorithmic complexity of such approaches make them less conducive to hardware acceleration on devices such as graphics processing units (GPUs), field programmable gate arrays (FPGAs), and application specific integrated circuits (ASICs). We demonstrate the first implementation of the BH tree on an FPGA in which the entire tree construction and walking is performed on the board. This required developing a novel BH tree algorithm to better match the FPGA architecture. Our tree construction and walking kernels are both pipelined with initiation intervals of 1. While the relative lack of progress in FPGAs over the last decade means our current algorithm is not competitive with current CPUs and GPUs, it does lay the groundwork for tree-specific ASIC development along the lines of GRAPE boards. Our source code is publicly available.

Keywords: Galaxies (573) — Cosmology (343) — High Energy astrophysics (739) — Interstellar medium (847) — Stellar astronomy (1583) — Solar physics (1476)

1. BACKGROUND

Many astrophysical systems like stars, galaxies, and the universe have significant self-gravity, and so modeling their evolution in numerical simulations requires solving for the gravitational force at an arbitrary position in space. Such systems are discretized, with a continuous distribution of mass divided amongst a finite number of N bodies typically treated as softened point masses at positions $\mathbf{x}_j$. The direct, or exact, computation of the force at a position $\mathbf{x}$ is then given by,

$$\mathbf{a}(\mathbf{x}) = \sum_{j=1}^N Gm_j W(|\mathbf{x}_j - \mathbf{x}|) \hat{r}, \quad (1)$$

where $\hat{r} = (\mathbf{x}_j - \mathbf{x}) / |\mathbf{x}_j - \mathbf{x}|$, and $W(r)$ is the softening kernel. In the Newtonian, non-softened case $W(r) = 1/r^2$.

To compute the self-gravity of such a system (i.e., to compute the force at the position of each of the N particles) requires the computation of $N(N-1)/2$ non-

trivial interactions, each of which involves an expensive `sqrt` call. This $\mathcal{O}(N^2)$ makes the direct computation of the gravitational force prohibitively expensive for systems involving more than $\sim 10^6$ particles.

As a result, many approximate methods for gravity calculations have been developed with much more attractive scalings. The first is the Barnes-Hut (BH) tree algorithm (J. Barnes & P. Hut 1986). In this method, the particle distribution is first divided into a hierarchical oct-tree (a quad-tree in 2D). This tree is subdivided until each “level” of the tree has one particle.⁴ The multipole moments of each oct-tree node are computed. To compute the acceleration at a position, one assesses the accuracy of the multipole approximation for each top-level node. If this accuracy is deemed insufficient, that node is “opened,” meaning all of its subnodes are assessed in the same way. This process is repeated until all nodes have been exhausted.

⁴ Variations allow for the node division to end when a certain number of particles (e.g., 8) are in each node, or only subdividing up to a maximum depth.

For a single force calculation, the number of interactions at each level of the tree remains roughly constant, and so the typical cost scales with the depth of the tree, which scales as $\log N$. Thus, for a system of particles, the total force calculation scales as $\mathcal{O}(N \log N)$. In astrophysics, the BH tree is widely adopted, underlying a substantial fraction of recent work in galaxy formation and cosmology (e.g., P. F. Hopkins et al. 2018; A. Pillepich et al. 2018; R. Davé et al. 2019).

An alternative technique is the fast multipole method (FMM; L. Greengard & V. Rokhlin 1987; H. Cheng et al. 1999). Here, instead of evaluating particle-node interactions, node-node pairs expand each other’s multipole moments locally. These local expansions are accumulated for all distant nodes. The force on each particle is evaluated only once after all distant nodes have been accounted for. This leads to an attractive scaling of $\mathcal{O}(N)$, and it has been applied in astrophysics (W. Dehnen 2000, 2014; V. Springel et al. 2021; M. Schaller et al. 2024) but at the cost of a higher algorithmic complexity.

A tree or FMM solver is often augmented with particle-mesh (PM) gravity solvers (R. W. Hockney & J. W. Eastwood 1981; A. A. Klypin & S. F. Shandarin 1983; S. D. M. White et al. 1983; V. Springel 2005; M. Vogelsberger et al. 2020). In the PM method, the discrete mass distribution is placed onto a grid through some kernel function, Fourier transformed, multiplied by the relevant Green’s function, and transformed back to real space to recover an approximate field potential. This potential can then be differentiated through standard methods in order to recover the force at a given point. While this technique can readily make use of the Fast Fourier Transform, a prohibitively fine grid is needed for the highly clustered systems common in astrophysics. For this reason, a short-range and long-range split is used, where contribution from long-range sources is handled by PM and from short-range forces by a tree, FMM, or direct summation method (G. Xu 1995; V. Springel 2005; L. H. Garrison et al. 2021).

Significant effort has been made to accelerate force calculations on general purpose hardware. Direct summation is straightforward to efficiently implement on GPUs (e.g., S. F. Portegies Zwart et al. 2007; L. Wang et al. 2015).

The BH tree does not map as cleanly to GPUs

This work is organized as follows. In Section 2, we summarize the existing BH tree implementations. In Section 3, we detail our new tree walking algorithm and explain why it is better suited to custom hardware than the existing implementations. Then, in Section 4 we discuss more of the technical details of how the algorithm

was implemented on our FPGA. In Section 5 we summarize the performance characteristics of our design, before concluding in Section 7.

2. CURRENT IMPLEMENTATIONS

We now first describe the algorithm in general and then summarize two of the more common current implementations. Implementations of the algorithm mainly differ in how the tree data is constructed and stored in memory.

2.1. Barnes-Hut Algorithm

The main idea behind the BH algorithm is to account for the partial force of distant particles by aggregating their effect together (J. Barnes & P. Hut 1986). In particular, one divides the domain into an octree (quadtree in 2D) and computes the multipole moments (monopole, quadrupole, etc.) of all particles in each spatial subdivision at each level. These subdivisions are referred to as nodes.⁵ This can be done down to a fixed maximum depth, or the spatial domain can be arbitrarily divided until each node contains fewer than a threshold number of particles (typically ~ 1 –10). Nodes that are not subdivided further (i.e., they have no children) are referred to as leaves.

Once the tree is built, forces are computed at a given point by recursively “walking” the tree. Starting at the top-level node, one either accepts the multipole approximation and adds its partial force contribution, or rejects it and applies the same consideration to all of its children nodes. This, of course, necessitates a so-called multipole acceptance criterion (MAC). The original MAC was a geometric one, in which the opening angle $\theta = l/d$ is computed, where l is the size of the node and d is the distance to the node’s center. If this angle is less than a threshold value (typically ~ 0.1 –0.7), then the node’s multipole approximation is accepted.⁶ Accepting the MAC for a node is referred to as closing the node, and rejecting it is referred to as opening the node.

Note that the algorithm is inherently irregular during tree walking, as it cannot be determined in advance which nodes will be opened or closed, and the precise

⁵ A node’s subdivisions are its children, nodes at the same level are siblings, and the node they are contained within one level higher is their parent.

⁶ Subsequently, a relative opening criterion was proposed by V. Springel et al. (2001) comparing the truncation error computed from the next-highest multipole term to an estimate of the force’s magnitude (e.g., from the previous timestep). For a given force accuracy requirement, this opening criterion requires less computational effort. See also J. K. Salmon & M. S. Warren (1994) for a discussion of worst-case error estimates from various opening criteria.

path will vary based on the target position. Furthermore, depending on the layout of the tree, node closures can result in quite large jumps in memory.

2.2. Pointer Chasing

Early implementations of the BH tree, and many subsequent codes, used pointer chasing.⁷ The tree is a pointer-based octree, where each node stores a pointer to its first child (if it has one) and to the next encountered node in a depth-first scan (e.g., its next sibling).

The tree topology is constructed via iterative walking. If the leaf threshold is 1, the tree is walked until an empty child slot is found. If, instead, a leaf already containing a particle is found, then that node is split and, if necessary, its child containing the two particles is split, until each particle sits in a unique node. This can be generalized if the desired leaf threshold is greater than 1.

Once the topology has been fixed, an upward pass computes the multipole moments of the internal nodes.⁸ This can be done in a number of ways, but one illustrative example is a postorder recursive traversal. Starting from the root, the routine descends through each node's children until it reaches the leaves, whose moments are computed directly from the particles they contain. As the recursion returns, each internal node combines the already-computed moments of its children to form its own multipole expansion. Applying this routine to the root therefore computes the moments of every node in the tree, proceeding implicitly from leaves to root.

Walking is then done simply by iteratively applying the MAC to each node, starting with the root node. The pointer to a node's first child is dereferenced when a node is opened, and the pointer to the next node in the depth-first scan is dereferenced when a node is closed.

The pointer-chasing technique is ill-suited to custom hardware solutions because of its irregular memory access pattern in both construction and walking. Of course, as mentioned before, the tree algorithm will always have some inherent irregularity in its memory access during walking. But in this case, the tree is repeatedly walked during the construction of its topology. Furthermore, the algorithm's branching logic presents additional problems.

⁷ See also <https://home.ifa.hawaii.edu/users/barnes/treecode/treeguide.html>

⁸ One can also avoid the upward pass by iteratively updating each node's multipole moments during the particle insertion phase.

2.3. Breadth-First

The incremental insertion of the pointer chasing technique is poorly suited to GPU-parallelization, which has motivated the technique proposed in *bonsai* by J. Bédorf et al. (2012). The tree is stored breadth-first, where each level is contiguous in the array. Each node stores the index of its first child and its number of children, along with the usual multipole moments.

First, all particles are sorted along a SFC like PH. In this order, particles belonging to the same node are always contiguous. Then, starting with the top level, each particle's SFC key is masked to identify the node that particle belongs to on that level. Particles are grouped by the node they occupy, and a list of index ranges is produced for that level for each node. Any node with fewer than a specified number of particles (N_{leaf}) is marked as a leaf and ignored at the next level. This process is repeated and the maximum level found.

Then, a linking stage is performed. Every node is assigned a thread and that thread finds the node's parent with a binary search algorithm, incrementing the parent's child count. The same thread also finds that node's first child node, also with a binary search.⁹ After this linking stage, a bottom up pass is performed to compute multipole moments.

Walking the tree is then performed with an interaction list.

Atomic locking operations are needed to then collate each node's properties. Particles belonging to leaf nodes, which contain fewer than a specified limit (e.g., 16), are marked as complete. The process iterates until all particles are complete or a specified maximum depth has been reached.

After the node properties have been computed, they must be linked together. Each cell is assigned to a thread and its parent and first child (if it is not a leaf) are found via a binary search algorithm, enabling tree walking.

3. FLATTENED TREE ALGORITHM

Neither pointer-chasing nor breadth-first tree algorithms are well-suited to FPGAs. Pointer-chasing trees require dynamic memory allocation and random access during both construction and traversal. The breadth-first algorithm of Bédorf requires multiple sorting passes, complicated linking logic, and atomic locking operations whose area requirements are prohibitive on FPGAs. Here we describe a new tree construction and traversal

⁹ By virtue of the particle list being initially sorted along a SFC, all of a node's children are contiguous. Therefore, to find all of a node's children requires only knowing the total number of children and the index of the first child.

sal algorithm designed around the streaming, pipelined execution model natural to FPGAs.

3.1. Tree Structure

Our tree is stored as a depth-first linear array of node-leaves. Because we mix nodes and leaves in the array, we use a common struct that can represent either a node or leaf. The first element is one of the top-level nodes, which we fix at level 1 (i.e., the first subdivision of the domain into octants). The next element is that node's first child, then that child's first child, and so on down to a user-specified maximum depth D . All nodes at depth D are leaves, and every leaf contains exactly one particle. If two particles share the same Peano–Hilbert (PH) key down to depth D , they occupy separate leaves with identical PH keys. In this way, every particle's position is stored directly in the tree.

Duplicating particle data in the leaves introduces a modest storage overhead but improves data locality during traversal, since particle and node data are interleaved in a single contiguous array.

3.2. Construction

Input: sorted particles P , max depth D
Output: tree array T (depth-first order)
 Initialize node stack $S[1..D]$ with $P[0]$;
for $i \leftarrow 1$ **to** $|P|$ **do**
 $\ell^* \leftarrow$ shallowest level where $P[i]$ diverges from S ;
 if no divergence **then**
 $\ell^* \leftarrow D$; // force new leaf
 end
 Emit $S[\ell^*..D]$ to T (deepest first);
 Reset $S[\ell^*..D]$ with $P[i]$;
 Accumulate $P[i]$ into $S[1..\ell^*-1]$;
end
 Flush $S[1..D]$ to T ;
 Reverse T ;

Algorithm 1: Tree construction

FPGAs achieve high throughput through pipelining: conceptually, data streams through the device rather than being acted upon in place. Our construction algorithm is designed with this model in mind.

Sorting.—Particles are first sorted by their PH key, computed to the maximum depth D . A key property of PH ordering is that a particle's key encodes its membership in every node from the root down to depth D . For example, in 2D a key 00 01 10 00 indicates membership in the 00 node at level 1, the 00 01 node at level 2, and so on. Consequently, all particles belonging to any given node are contiguous in the sorted order.

Streaming accumulation.—A stack of D nodes is maintained in local memory. Each node tracks accumulated center-of-mass and multipole data. The stack is initialized with the properties of the first particle.

Particles are then processed sequentially. For each new particle, its PH key is compared against the keys of the nodes on the stack to determine the *divergence level*: the shallowest level at which the new particle's key differs from the current stack. Nodes shallower than the divergence level (i.e., those the new particle still belongs to) continue accumulating. All nodes at or deeper than the divergence level are finalized: they are emitted and appended to the output array, then reset and seeded with the new particle's properties. After the final particle has been processed, the remaining stack is flushed.

Sibling connections (i.e., the index of each node's next sibling) are recorded during emission, so no separate linking pass is required. Note that for leaf nodes, the next sibling is always just the next node in the list.

Output.—The result is a linear array of nodes in reverse depth-first order: the last node emitted is a top-level node. The array is reversed to yield the final tree. Because the tree is constructed in streaming fashion, it is also possible to begin traversal before construction is complete—for example, once one of the eight top-level octants has been fully emitted—though we leave this optimization to future work.

3.3. Walking

Input: particles P , tree nodes T
Output: accelerations $\mathbf{a}$
foreach particle $p \in P$ **do**
 $i \leftarrow 0$;
 $\mathbf{a}_p \leftarrow 0$;
 while $i \neq \text{SENTINEL}$ **do**
 $n \leftarrow T[i]$;
 if accept(p, n) **or** n .is.leaf **then**
 $\mathbf{a}_p \leftarrow \mathbf{a}_p + \text{force}(p, n)$;
 $i \leftarrow n.\text{next_sibling}$; // node closed
 else
 $i \leftarrow i + 1$; // node opened
 end
 end
end

Algorithm 2: Tree walk

Tree walking is a relatively straightforward operation, outlined in Algorithm 2. For each particle, we start with the first top-level node (the last node to be emitted during the tree construction process), the desired multipole acceptance criterion is evaluated for the particle-node pair. If the criterion is accepted or the node is a leaf,

the particle accumulates the node’s force contribution and the loop skips to the next sibling, whose index is stored with the node’s data. If the criterion is rejected, the loop continues to the next index, which is guaranteed to be that node’s first child. The process is iterated until all nodes have been considered.

4. DESIGN

We now summarize the details of how we have implemented our new BH tree algorithm on our hardware.

4.1. Hardware Setup

Our design targets an AMD Alveo U200 (hosting the XCU200-L2FSGD2104E device) and runs at a post-route clock frequency of 100 MHz. This board has 64 GB of DDR4 RAM with a transfer rate of up to 2400 MT/s across four banks. Our source code is written in pure C++ HLS, which we compile into a bitstream using AMD’s Vitis HLS toolchain version 2023.2.2. For continuous integration, we built a Docker image with the Vitis HLS toolchain installed. We cannot make this image public due to licensing requirements, but the steps we took to create it are documented in the `DOCKER_IMAGE.md` file. All tests were performed in a container based on this image.

Our host system, and the Docker image, runs Ubuntu 22.04.4 LTS. Our host has an Intel Xeon Silver 4216 CPU @ 2.10GHz with 16 cores (32 threads), and 128 GB of DDR4 ECC RAM running at 2400 MT/s. Our Alveo U200 is connected to the host over PCIe Gen3 at 8 GT/s across 16 lanes.

4.2. Data Types

We now summarize the salient data types of our design, with a full list given in Table 1. Because coordinates have values in $[0, 1)$, we store them in unsigned fixed point with N fractional bits and no integer bits, denoted `pos_t`. For this work we always assume $N = 32$, intermediate between the 23 and 52 mantissa bits of 32- and 64-bit floating point, respectively. This is equivalent to integer coordinates as in V. Springel et al. (2021).¹⁰ Storing coordinates in fixed point allows intermediate calculations to also use fixed precision with finite bounds, avoiding costly floating point operations. Particle and node masses are stored as unsigned fixed point fractions of the total box mass (`mass_t`), with convergent rounding and saturation overflow modes enabled

(since the total mass may slightly exceed unity due to roundoff error). In this work we use the same number of fractional bits N for both `pos_t` and `mass_t`.

Quantities derived from the softening length, however, can require quite wide data types if implemented naively. For example, if the softening length h were represented with the `pos_t` type, then $1/h^3$ would need a 128-bit representation, and additions and multiplications with such wide types are expensive. Floating point is the natural solution, but is itself resource intensive on hardware, largely because the IEEE-754 standard requires several features like normalization (keeping the mantissa in $[1, 2)$), rounding to even, and special case handling (NaN and infinity).

In our case, we require some of the dynamic range of floating point but not necessarily all of the IEEE-754 features and guarantees. Instead, we use a custom floating point architecture with similar precision to `float32`. All intermediate calculations are unnormalized and no special case checking is done, which is valid because we know the dynamic range of all quantities involved. We also truncate instead of rounding, carrying one additional bit of mantissa precision to give the same 0.5 ULP precision as `float32`. This leads to a much lower resource usage than standard floating point.

Rather than passing softening lengths as arbitrary numbers, we construct 64-entry softening tables—one for h^{-2} and one for h^{-3} —in which successive entries are spaced by a factor of 1.2. The softening length of each particle and node is then stored as a 6-bit unsigned integer key (`hidx_t`), and min/max comparisons can be done directly on the integer, which is significantly cheaper than fixed- or floating-point comparators. The table entries themselves are stored in our custom floating point format with a 6-bit unsigned exponent (unsigned since we always assume $h < 1$ and store inverses). This corresponds to a maximum value of h^{-3} of 2^{64} , or a minimum value of h of $\sim 3.8 \times 10^{-7}$ (in units of the box size). For reference, for a 50 Mpc box, this corresponds to a softening of ~ 19 pc. Smaller softenings can be achieved by widening the exponent.

Instead of directly computing the partial forces, which would require an inverse square root and many multiplications within the softened regime, we perform a direct interpolation. This means that, for a given particle-particle or particle-node interaction, we compute the partial force as

$$\mathbf{a}_{ij} = m_j h_{ij}^{-3} g(u_{ij}) (\mathbf{x}_j - \mathbf{x}_i), \quad (2)$$

where m_j is the mass of particle/node j , h_{ij} is the pairwise softening, u_{ij} is the distance between i and j normalized by the softening length, and $g(u)$ is the dimen-

¹⁰ SFC ordering keeps consecutive particles close in space, so the memory footprint could be reduced by storing only coordinate differences between successive particles, or via the more sophisticated scheme of H.-R. Yu et al. (2018).

sionless, softened force law. In GADGET notation (V. Springel 2005), our g is simply $-h^3 g_1$. Because computing u requires a square root, we tabulate g as a function of u^2 instead. We compute r^2 in fixed point and then convert to floating point; multiplying by the tabulated value of h^{-2} to obtain u^2 is then cheap (one mantissa multiplication and one exponent addition).

Our interpolation table for g stores values only for $u^2 \in [0, 4)$. The softened force law applies for $u^2 < 1$, and when $u^2 > 4$, we simply rescale to be within $1 < u^2 < 4$, using the fact that $g = 1/u^3$ for $u > 1$ to scale back at the end (both scalings are exponent manipulations). We then multiply by the mass of the particle/node and by the tabulated value of h^{-3} . The partial force is cast to an IEEE-754 `float32` and then accumulated. We normalize only when computing position differences, when computing u^2 , before multiplying by the node/particle mass, and at the end when casting to `float32`.

Note that the widths of the `phkey_t` and `level_t` data types depend on the chosen maximum depth D .

4.3. Dataflow Implementation

On the FPGA, this loop is implemented using a cyclic dataflow paradigm with five subkernels: `read_particles`, `work_distributor`, `force_kernel`, `read_kernel`, and `output_kernel`. These subkernels perform the following tasks:

- **read_particles:** Particles are read from DRAM and placed into a stream connected to `work_distributor`.
- **work_distributor:** This kernel places particles into the cyclic region by streaming them to `force_kernel`. Particles are read from a stream from `read_particles` (new particles) and from a stream from `read_kernel` (particles existing in the cyclic region). If a particle is available from both streams, priority is given to particles that are continuing in the region.
- **force_kernel:** Particles are read from a stream from `work_distributor`. The opening criterion is evaluated and, if necessary, the partial force contribution is accumulated. The particle, along with the next node that needs to be read, is passed to `read_kernel`. If the particle has reached the end of the tree, it is instead passed to `output_kernel`.
- **read_kernel:** Particles are accepted from `force_kernel` along with their next requested node. A DRAM request is made for this node and, when fulfilled, the particle is sent back to `work_distributor`.

- **output_kernel:** Particles from `force_kernel` with their completed accelerations are written to a DRAM buffer for the host to retrieve.

Each subkernel is pipelined with an II of 1, and so one interaction can be processed per clock cycle if the dataflow region can be continually fed. The total pipeline has a latency of 300 clock cycles **TODO: finalize number**, and so we need to have $\gtrsim 300$ particles in flight at a time in order to keep the pipeline full.

This walking algorithm is better suited to FPGAs than prior methods because it limits random memory accesses to just node closures – any time a node is opened or a leaf is encountered, the next node is always the next entry in the tree.

4.4. Configuration

The U200 has three super logic regions (SLRs) and four DDR banks, with each bank physically close to one SLR. Crossing an SLR boundary requires traversing a super long line (SLL), which adds routing delay and is a common source of timing closure difficulties at high clock frequencies. We therefore place each kernel and its associated memory interfaces on a single SLR whenever possible.

We place the `treecon` kernel on SLR0 and the `treewalk` kernel on SLR1. Both kernels are clocked at 100 MHz. Three of the four DDR banks are used. The input particle buffer resides in DDR0 and is read by both kernels: first by `treecon` during streaming tree construction, and again by `treewalk` as the source of the particles whose accelerations are to be computed. The tree itself is written by `treecon` to DDR1 and is read back by `treewalk` during traversal. The output accelerations are written by `treewalk` to DDR2. DDR3 is unused.

This assignment is motivated by the asymmetry in memory traffic between the two kernels. The `treewalk` kernel is latency-bound on its tree AXI port and thus benefits most from a low-latency path to the tree, so we co-locate it with its heavily-used DDR bank on SLR1. This means that the `treecon` kernel crosses a SLL to write the tree. However, because the tree write is sequential and `treecon` is a subdominant component of the runtime anyways, this SLL crossing is tolerable. There is also an SLL crossing when `treewalk` reads from the particle list, but again this read is sequential and each particle requires multiple trips through the `treewalk` data loop, so the higher latency and lower bandwidth incurred by the SLL crossing is tolerable.

Table 1. Data types used in the force calculation.

Type	Definition	Description
pos_t	ap_ufixed<32,0>	coordinates
mass_t	ap_ufixed<32,0,AP_RND_CONV,AP_SAT>	mass (fraction of total)
acc_t	float	acceleration (output)
hidx_t	ap_uint<6>	softening table index
phkey_t	ap_uint<48>	Peano-Hilbert key
level_t	uint8_t	tree depth (max 64)
count_t	uint32_t	indices and counters
flag_t	ap_uint<1>	boolean flag

Table 2. Particle data structure (`particle_t`).

Field	Type	Width (bits)	Description
pos[3]	pos_t	96	position
acc[3]	acc_t	96	acceleration
mass	mass_t	32	mass fraction
idx	count_t	32	particle index
key	phkey_t	48	Peano-Hilbert key
next_tree_idx	count_t	32	next tree index
hidx	hidx_t	6	softening table index
Nint_opened	count_t	32	node-opening count
Nint_leaf	count_t	32	leaf interaction count
Nint_node	count_t	32	node interaction count
<i>padding</i>		74	
Total		512 (64 bytes)	

Table 3. Node/leaf data structure (`nodeleaf_t`).

Field	Type	Width (bits)	Description
pos[3]	pos_t	96	center of mass / first particle position
mass	mass_t	32	mass fraction
level	level_t	8	tree depth
start_idx	count_t	32	index of first child/particle
num_particles	count_t	32	particle count
next_sibling	count_t	32	next sibling index
is_leaf	flag_t	1	leaf flag
is_last	flag_t	1	last sibling flag
valid	flag_t	1	validity flag
hmax_idx	hidx_t	6	max softening table index
target_cell	count_t	32	target cell index (TODO: remove)
<i>padding</i>		239	
Total		512 (64 bytes)	

Table 4. Force calculation for particle p and node n . Intermediate quantities are held in our custom mantissa/exponent floating point format (denoted m/e), in which mantissas are unnormalized except where noted and no rounding or special case handling is performed until the final cast to `float32`.

Computation	Conversion	Note
$\Delta x \leftarrow x_n - x_p$	<code>pos_t</code> $\rightarrow$ fixed	also for Δy and Δz
$r^2 \leftarrow \Delta x^2 + \Delta y^2 + \Delta z^2$	fixed $\rightarrow$ m/e	normalized
$k \leftarrow \max(k_n, k_p)$	—	pairwise softening; 6-bit integer compare
$h^{-2}, h^{-3} \leftarrow \text{soft_table}[k]$	—	stored as m/e pairs
$u^2 \leftarrow r^2 \cdot h^{-2}$	—	exponent gives exact $u^2 < 4$ test
$g \leftarrow \text{g_lut}(u^2)$	—	1024-entry LUT + lin. interp.; u^2 is rescaled when $u^2 \geq 4$
$f \leftarrow m_n \cdot h^{-3} \cdot g$	<code>mass_t</code> $\rightarrow$ m/e	mass normalized first
$\mathbf{a}_p \leftarrow \mathbf{a}_p + f \cdot \Delta \mathbf{x}$	m/e $\rightarrow$ <code>float32</code>	$\Delta \mathbf{x}$ normalized; sign carried separately

4.5. Resource Usage

We show the final resource usage in Table 5 for our design. The main thing to note here is that the **treewalk** kernel’s largest usage is in DSPs, consuming 252 of the board’s 6840 DSPs. This implies that our **treewalk** design could be replicated on the board approximately 28 times, depending if further optimizations could be made, and approximately 50 times on the larger U250 board (which hosts the XCU250-L2FIGD2104E device).

5. PERFORMANCE

We now turn to summarizing the performance of our design. First, we will demonstrate the accuracy of our design with respect to a direct sum calculation.

We test our design on two setups with varying particle numbers and distributions: a uniform density distribution and a Hernquist sphere, the latter which intends to test the design on a clustered distribution. In each case, we vary the particle number by factors of 4 from 2^{12} to 2^{20} (from $\sim 4k$ to $\sim 1M$). For the Hernquist sphere, we also test varying its scale length a , varying from 0.001 to inf (i.e., a uniform sphere of radius 0.5).

5.1. Accuracy

We show in Figure 5.1 the relative force error as measured between the FPGA computed value and the value from direct summation. The mean and 90th percentile values computed from a random subsample of 4096 particles are shown in solid and dashed lines, respectively.

In general, the force accuracy is better in the Hernquist sphere case than for the uniform distribution. The maximum and median errors in the Hernquist sphere are no larger than $\sim 2\%$ and $\sim 0.3\%$, respectively. For the uniform distribution these values are instead $\sim 10\%$ and $\sim 0.4\%$. One difference is that in the uniform distribution case the maximum error improves at larger particle number while the mean error remains roughly constant. However, in the Hernquist case the opposite is true: the maximum error is relatively constant while the mean error declines with increasing particle number.

In Figure 5.1, we show how the relative force error scales with opening angle θ . Because the dipole moment vanishes, the leading order error term is expected to be $\propto \theta^3$ (e.g. V. Springel et al. 2021), which we plot as a dashed line. The 90th percentile relative error follows this scaling reasonably well, with the errors being about twice as large at the low- θ end. **TODO: Figure out why.**

We have also verified that our FPGA and software implementations give nearly identical acceleration values. For the same 4096 particle subsample, the maximum deviation between the two implementations is $\sim 0.27\%$ or $\sim 0.02\%$ in the uniform/Hernquist sphere cases, respec-

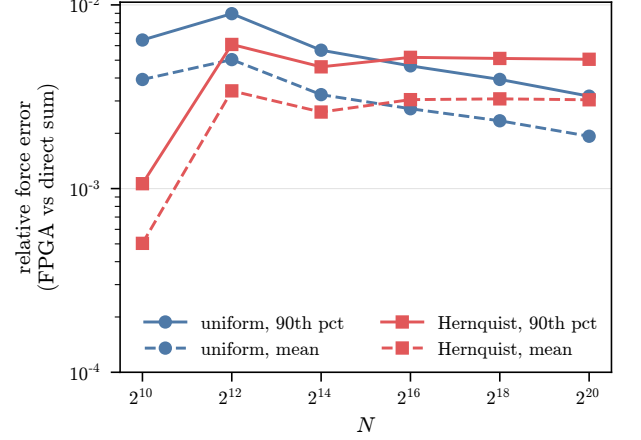


Figure 1. Error scaling with particle number for our uniform (blue circles) and Hernquist ($a = 0.5$; red squares) distributions. We show the 90th percentile and mean errors in solid and dashed lines, respectively. These relative errors statistics, computed against a direct summation estimate, all lie below 1%.

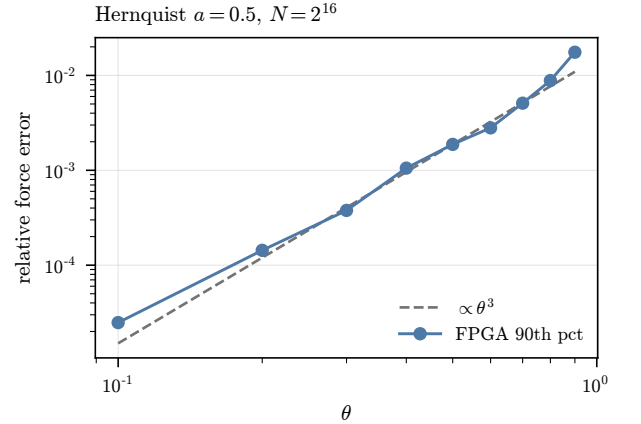
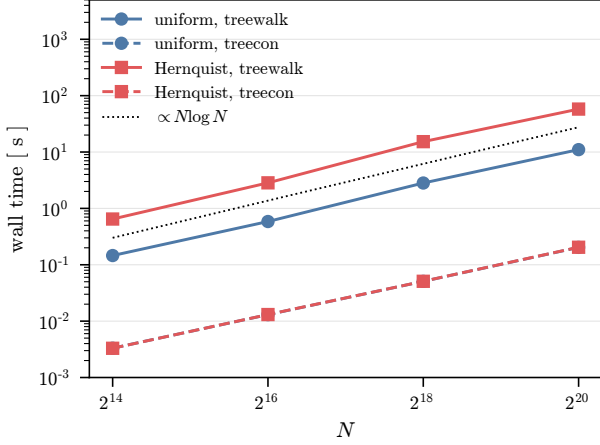


Figure 2. The 90th percentile relative error as a function of opening angle θ , for the distribution with $a = 0.5$ and 2^{16} particles ($\sim 65k$). The error value follows the expected θ^3 scaling (gray dashed line anchored to $\theta = 0.5$) except close to $\theta = 1$ and $\theta = 0.1$. At the lower end, the deviation results from the FPGA’s use of a fixed-point representation (with 32 fractional bits) for the particle positions, while the direct summation reference calculation uses a double representation, which has no fewer than 53 fractional bits between 0 and 1.

tively. In both cases, 99% of particles have a relative difference between the two of less than 2.8×10^{-5} .

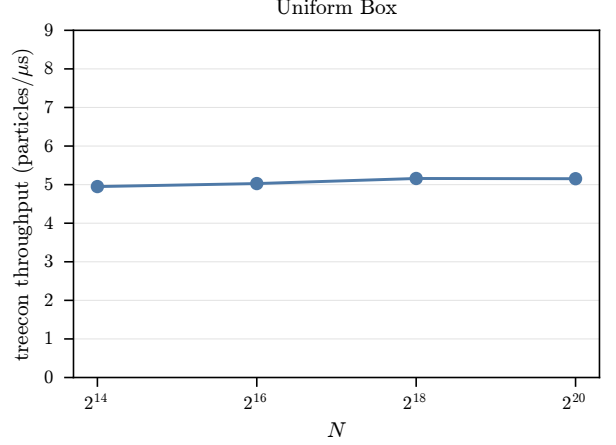
Table 5. Post-route resource utilization on the Alveo U200 for the combined dual-kernel design. **TODO: Finalize these numbers.**

	LUT	FF/REG	BRAM	DSP
treecon	35 549 (3.70%)	30 624 (1.50%)	13 (0.72%)	19 (0.28%)
treewalk	24 711 (2.57%)	18 254 (0.89%)	70 (3.88%)	233 (3.41%)
Kernel total	60 260 (6.28%)	48 878 (2.39%)	83 (4.61%)	252 (3.69%)
Full design	282 754 (23.92%)	368 756 (15.60%)	441.5 (20.44%)	265 (3.87%)
Available	1 182 240	2 364 480	2 160	6 840

**Figure 3.** The scaling of our algorithm with problem size. The wall time for the This is shown for the uniform and Hernquist $a = 0.5$ distributions in blue circles and red squares, respectively. The **treewalk** and **treecon** kernels are shown in solid and dashed lines, respectively. Note that the **treecon** lines almost completely overlap between the two distributions. **TODO: Make treewalk/treecon in figure tt. Add $N = 2^{12}$ and 2^{10} ?**

5.2. Total Wall Time

Having demonstrated the accuracy of our FPGA implementation, we now turn to its performance. In Figure 5.2. We show the total wall time spent on the **treewalk** and **treecon** kernels in solid and dashed lines, respectively. As before, the uniform and Hernquist $a = 0.5$ distributions are shown in blue circles and red squares, respectively. These are all shown for varying particle number N . Note that the **treecon** lines are nearly overlapping for the two distributions, indicating that our **treecon** algorithm’s performance is independent of the particle distribution. The **treewalk** kernel, on the other hand, does depend on the particle distribution, with the more clustered Hernquist distribution taking about four times as long as the uniform distribution. All four lines closely following the expected $N \log N$ scaling, shown as a black dotted lines.

**Figure 4.** The scaling of **treecon** with particle number for the uniform box. At all particle numbers, the throughput saturates at around 5 particles μs^{-1} , close to the theoretical maximum of 6.25 particles μs^{-1} . **TODO: Add 6.25 line.**

5.3. Tree Construction Throughput

We show the throughput for the **treecon** kernel in Figure 5.3 for the uniform distribution scaling with particle number. The total throughput has a theoretical maximum of 6.25 particles μs^{-1} , given that the kernel runs at a frequency of 100 MHz and has an II of 16. The design achieves a throughput of about 5 particles μs^{-1} . The high II is due to the fact that the node emission loop is serialized as opposed to unrolled. This saves resources at the price of performance, but given that the wall time of **treecon** is subdominant to the wall time of **treewalk** (see Figure 5.2), this tradeoff is warranted in our case.

5.4. Tree Walking Throughput

We next show the throughput for the **treewalk** kernel in Figure 5.4. On the left panel we show the uniform distribution and on the right panel we show the Hernquist $a = 0.5$ sphere. In this case, our II is 1, and so the maximum throughput is 100 MHz, shown as a horizontal dashed line. We show the average number of node interactions, leaf interactions, and node openings per μs as blue squares, orange triangles, and green diamonds, re-

spectively. The total throughput, combining these three numbers, is shown as black circles.

First, the total maximum throughput is high, achieving $> 90\%$ of the maximum for nearly all particle numbers. In both distributions, the number of node interactions increases as the number of leaf interactions decreases.

In Figure 5.4, we extend the discussion by varying the scale length of the Hernquist distribution. The upper pnael shows the throughput of the design, where the various lines are the same as in Figure 5.4. The important thing to take away from this figure is that the maximum throughput remains high even as the particle distribution becomes more concentrated ($a \rightarrow 0$). In the bottom panel, we also show that the more concentrated distribution has a more accurate force estimate, which could be explained by the higher fraction of leaf interactions compared to node interactions shown in the upper pnael.

6. DISCUSSION

6.1. Comparison to CPUs and GPUs

We showed in Section 5.4 that our FPGA implementation is able to achieve $\gtrsim 90\%$ of the theoretical maximum throughput of $100 \text{ it}/\mu\text{s}$ (where interactions includes node openings). We tested our CPU implementation of the same algorithm and found that our hardware is able to reach $\sim 125 \text{ it}/\mu\text{s}$.¹¹ A competitive comparison between the two thus comes down to how many **treewalk** kernels can be replicated. **TODO: awk**

Our **treewalk** design is resource-limited by its DSP usage.¹² The **treecon** kernel and shell add very little DSP usage, and so this implies 28 additional kernels could fit onto the board. Some overhead needs to be left for the compiler, and so we assume ~ 24 additional kernels can fit onto the U200. The larger U250 board has about twice as many DSP units, and so we assume ~ 50 **treewalk** kernels could fit on it.¹³

¹¹ We tested using `gcc v16.1.0` on our local desktop in addition to `gcc v15.2.0` on the Torch cluster at New York University. Our desktop has an Intel Xeon Silver 4216 CPU @ 2.10GHz (hyperthreading enabled) and the node we tested on with Torch has an Intel Xeon Platinum 8592+ CPU (hyperthreading disabled).

¹² The **treewalk** kernel uses 70 BRAM blocks which is a higher fraction of the BRAM usage than DSPs. However, most of the BRAM usage comes from FIFOs between dataflow regions, and so can easily be absorbed into the available URAM blocks.

¹³ Of course, these make the assumption that all 25 or 50 kernels could be kept fed by the available DRAM banks. This may necessitate a custom memory controller/cache to limit the number of DRAM requests, which would reduce the number of resources available to any computational units.

On the CPU side, modern clusters have nodes with core counts exceeding 128. Assuming the optimistic case for CPUs, a single node with 128 cores operating at $\sim 125 \text{ it}/\mu\text{s}$ and achieving perfect parallelism would be capable of a throughput of $1.6 \times 10^4 \text{ it}/\mu\text{s}$. In order to match this performance, we would need 160 **treewalk** kernels. If we pushed our design to $300 \text{ it}/\mu\text{s}$ (the maximum clock speed of the U200 board), we would only need 53 **treewalk** kernels, which is comparable to what the U250 can deliver.

TODO: Add GPU comparison, ask Marco.

6.2. Future Improvements

In this work, we have focused on a proof-of-concept implementation of our hardware design. We have shown that our design is accurate and meets our performance goals. Of course, there are several areas for future improvement and refinement, which we discuss now.

First, the design's clock speed could be increased. We targeted 100 MHz, while the maximum allowed speed is 300 MHz. Increasing the clock speed typically requires a longer compilation time, and so we did not pursue this in this work.

Second, multiple **treewalk** (or **treecon**) kernels could be placed on the board. This would introduce several challenges, especially if multiple kernels needed to be connected to the same DRAM bank. In that case, a slightly more sophisticated memory sharing architecture could be warranted to limit irregular memory accesses.

Third, in our design sorting along the SFC is done on the host. In target cases where gravity is coupled to, e.g., hydrodynamics, this is probably not a huge issue since the particle data needs to be transferred onto and off of the board anyways. However, for a pure gravity simulation, it would be desirable to do this sorting on the board to eliminate this communication. Efficient sorting algorithms for FPGAs already exist (e.g., radix sorting), and given the specific drift pattern of astrophysical simulations, custom sorting algorithms may be beneficial.

Fourth, our streaming construction and walking paradigm could be relatively straightforwardly extended to the FMM. This would require more sophisticated control logic within the **treewalk** kernel as now chunks of the particle array would need to be streamed, but this may actually be a better match to our design as it increases the streaming fraction of the data. **TODO: Think about this more.**

Fifth, **TODO: more?**

Because tree construction is more than an order of magnitude faster than walking in our experiments, we did not expend a large amount of effort beyond getting

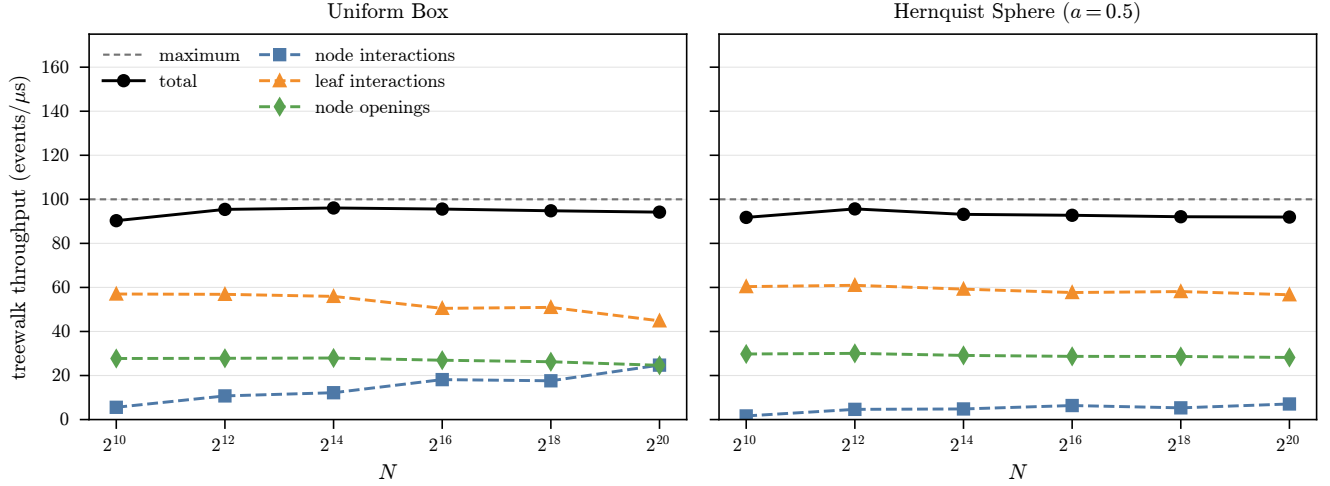


Figure 5. The total treewalk throughput as a function of particle number N for the uniform (left) and Hernquist $a = 0.5$ distributions. The solid black line shows the total throughput including interactions and node openings, which is always $> 90\%$ of the maximum of $100 \mu s^{-1}$. In both cases, as the particle number increases the number of node interactions increases and the number of leaf interactions decreases, although the effect is more subtle in the Hernquist sphere case. This is discussed in [TODO: Section X](#).

it running on the board with a hardware-friendly design. That being said, if many `treewalk` kernels were to be instantiated, then this may need to be revisited. It should be straightforward to parallelize `treecon`. This could be done by splitting the domain at an upper level and feeding particles from each of the upper level nodes to different `treecon` kernels. This splitting could be done at precise PH boundaries and the full tree stitched together, or the particle array could be split in chunks with uniform particle numbers. In fact, one could simply do a cyclic splitting of particles across the `treecon` kernels. For instance, suppose 2 kernels have been made. Particles 0, 2, etc. goes to kernel 0, and particles 1, 3, etc. goes to kernel 1. Each of these kernels create their own trees. A separate kernel could then intake two input trees and emit the combined tree. If 4 `treecon` kernels are desired, then 3 of these combining kernels could be chained together. **TODO: Make this better lol.**

In this work we have focused on the Newtonian gravity case. However, in cosmological simulations one must also account for the periodicity of the box. In a tree-only mode, this is typically accounted for by means of Ewald summation in the form of a tabulated correction table (L. Hernquist et al. 1991; P. P. Ewald 1921). For example, in GADGET-2 this was done using three 65^3 trilinear interpolation tables. These tables would require 68 URAM blocks if each entry were allocated 24 bits, and so 11 copies of the table could fit on the board. One could feed all kernels by sharing the blocks. For example, one could have three banks of 23 URAM blocks. Six kernels could be served by these 69 blocks by hav-

ing two read from the first bank, two from the second, and two from the third, cycling across three clock cycles. This would simply cost two additional clock cycles of latency. In this way, 66 kernels could fit on the U200. This would entail a relatively large design change, and so one would first need to consider whether the tradeoff is worth it compared to simply using the hybrid TreePM method where the Ewald correction is not necessary.

In this work, we only consider a simple geometric opening criterion. As discussed in V. Springel (2005), a relative opening criterion in which the roundoff error is computed and compared to the acceleration from the previous timestep performs better in cosmological simulations where accelerations can vary by several orders of magnitude. Our method could be extended to include this criterion with a relatively modest expansion in board area since, for our monopole approximation, it requires four multiplications – two to compute r^{-4} and two to multiply by M and l^2/α . Further, since this approximation only impacts the decision to open or close a node, it is not necessary to store the old acceleration or do this computation in 32-bit floating point, or potentially shorter bit-widths pending a numerical precision analysis.

7. CONCLUSIONS

We have introduced the first, to our knowledge, implementation of the Barnes-Hut gravity tree on an FPGA in which both tree construction and walking are performed on the board.

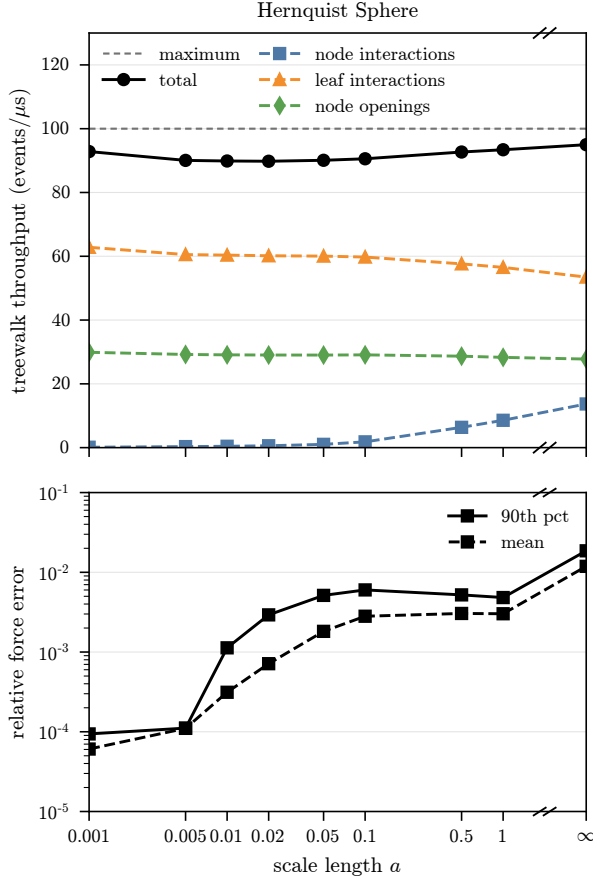


Figure 6. Error and `treewalk` throughput for the Hernquist sphere scaling with scale radius a . The upper panel shows the total throughput (black solid line) as well as the various components in the same style as Figure 5.4. As the distribution becomes more concentrated ($a \rightarrow 0$), the number of leaf interactions relative to the number of node interactions declines. This leads to a more accurate force estimate (shown as the mean and 90th percentile relative errors in the solid and dashed lines in the lower panel). At all scale lengths, the total throughput remains close to the maximum throughput.

Our implementation of the BH algorithm requires a strict ordering along a digit-recursive space filling curve, such as Morton Z-order or Peano-Hilbert. Then, tree construction occurs as a single streaming pass through the particles. The tree is then stored as a linear, depth-first array. For a given particle, walking is a one-way traverse through this array with skips during node closures (a MAC acceptance).

Given that FPGA development has not been commensurate with CPU and GPU advances, it is unlikely that an FPGA solution will compete with general purpose hardware in the near future. However, our novel design is a necessary advance for future development of gravity trees fully native on ASICs.

ACKNOWLEDGMENTS

Ack.

AUTHOR CONTRIBUTIONS

All authors contributed equally to the Terra Mater collaboration.

APPENDIX

A. APPENDIX

REFERENCES

- Barnes, J., & Hut, P. 1986, *Nature*, 324, 446, doi: [10.1038/324446a0](https://doi.org/10.1038/324446a0)
- Bédorf, J., Gaburov, E., & Portegies Zwart, S. 2012, *Journal of Computational Physics*, 231, 2825, doi: [10.1016/j.jcp.2011.12.024](https://doi.org/10.1016/j.jcp.2011.12.024)
- Cheng, H., Greengard, L., & Rokhlin, V. 1999, *Journal of Computational Physics*, 155, 468, doi: [10.1006/jcph.1999.6355](https://doi.org/10.1006/jcph.1999.6355)
- Davé, R., Anglés-Alcázar, D., Narayanan, D., et al. 2019, *MNRAS*, 486, 2827, doi: [10.1093/mnras/stz937](https://doi.org/10.1093/mnras/stz937)

- Dehnen, W. 2000, ApJL, 536, L39, doi: [10.1086/312724](https://doi.org/10.1086/312724)
- Dehnen, W. 2014, Computational Astrophysics and Cosmology, 1, 1, doi: [10.1186/s40668-014-0001-7](https://doi.org/10.1186/s40668-014-0001-7)
- Ewald, P. P. 1921, Annalen der Physik, 369, 253, doi: [10.1002/andp.19213690304](https://doi.org/10.1002/andp.19213690304)
- Garrison, L. H., Eisenstein, D. J., Ferrer, D., Maksimova, N. A., & Pinto, P. A. 2021, MNRAS, 508, 575, doi: [10.1093/mnras/stab2482](https://doi.org/10.1093/mnras/stab2482)
- Greengard, L., & Rokhlin, V. 1987, Journal of Computational Physics, 73, 325, doi: [10.1016/0021-9991\(87\)90140-9](https://doi.org/10.1016/0021-9991(87)90140-9)
- Hernquist, L., Bouchet, F. R., & Suto, Y. 1991, ApJS, 75, 231, doi: [10.1086/191530](https://doi.org/10.1086/191530)
- Hockney, R. W., & Eastwood, J. W. 1981, Computer Simulation Using Particles
- Hopkins, P. F., Wetzel, A., Kereš, D., et al. 2018, MNRAS, 480, 800, doi: [10.1093/mnras/sty1690](https://doi.org/10.1093/mnras/sty1690)
- Klypin, A. A., & Shandarin, S. F. 1983, MNRAS, 204, 891, doi: [10.1093/mnras/204.3.891](https://doi.org/10.1093/mnras/204.3.891)
- Pillepich, A., Springel, V., Nelson, D., et al. 2018, MNRAS, 473, 4077, doi: [10.1093/mnras/stx2656](https://doi.org/10.1093/mnras/stx2656)
- Portegies Zwart, S. F., Belleman, R. G., & Geldof, P. M. 2007, NewA, 12, 641, doi: [10.1016/j.newast.2007.05.004](https://doi.org/10.1016/j.newast.2007.05.004)
- Salmon, J. K., & Warren, M. S. 1994, Journal of Computational Physics, 111, 136, doi: [10.1006/jcph.1994.1050](https://doi.org/10.1006/jcph.1994.1050)
- Schaller, M., Borrow, J., Draper, P. W., et al. 2024, MNRAS, 530, 2378, doi: [10.1093/mnras/stae922](https://doi.org/10.1093/mnras/stae922)
- Springel, V. 2005, MNRAS, 364, 1105, doi: [10.1111/j.1365-2966.2005.09655.x](https://doi.org/10.1111/j.1365-2966.2005.09655.x)
- Springel, V., Pakmor, R., Zier, O., & Reinecke, M. 2021, MNRAS, 506, 2871, doi: [10.1093/mnras/stab1855](https://doi.org/10.1093/mnras/stab1855)
- Springel, V., Yoshida, N., & White, S. D. M. 2001, NewA, 6, 79, doi: [10.1016/S1384-1076\(01\)00042-2](https://doi.org/10.1016/S1384-1076(01)00042-2)
- Vogelsberger, M., Marinacci, F., Torrey, P., & Puchwein, E. 2020, Nature Reviews Physics, 2, 42, doi: [10.1038/s42254-019-0127-2](https://doi.org/10.1038/s42254-019-0127-2)
- Wang, L., Spurzem, R., Aarseth, S., et al. 2015, MNRAS, 450, 4070, doi: [10.1093/mnras/stv817](https://doi.org/10.1093/mnras/stv817)
- White, S. D. M., Frenk, C. S., & Davis, M. 1983, ApJL, 274, L1, doi: [10.1086/184139](https://doi.org/10.1086/184139)
- Xu, G. 1995, ApJS, 98, 355, doi: [10.1086/192166](https://doi.org/10.1086/192166)
- Yu, H.-R., Pen, U.-L., & Wang, X. 2018, ApJS, 237, 24, doi: [10.3847/1538-4365/aac830](https://doi.org/10.3847/1538-4365/aac830)